Discussion 7: Comparing Edge Detection Methods

Discussion Topic:

Comparing Edge Detection Methods

Edge detection is a crucial stage in numerous image processing applications. One of the most challenging goals in computer vision is to develop algorithms that can process visual information reliably. To ensure that an edge detection technique is reliable, it needs to be rigorously assessed before being used in a computer vision application.

Using OpenCV, generate a synthetic image that contains exactly one filled-in square and one filled-in circle. The placement and color intensities of these shapes are up to you. The background intensity is up to you as well. You should know precisely the locations of the discontinuities. The rest of the image should be without edges.

Implement Canny, Sobel, and then Laplacian edge detection on this image. Define a measure to evaluate the performance of each method. Repeat this experiment by adding noise to the image using a random number generator and changing the intensity values of the objects and the background. Change the threshold values for your detection step and then evaluate the performance once again.

In your post, discuss the following:

1. Describe the evaluation method you developed and the factors you considered in its definition.
2. Compare your evaluation method to at least two other edge detection evaluation methods you have researched.
3. Which edge detection method worked best under each of the conditions (original image vs adding noise and varying intensity values)? Substantiate your claim with evaluation method metrics.

Attach your synthetic image to your post.

My Post:

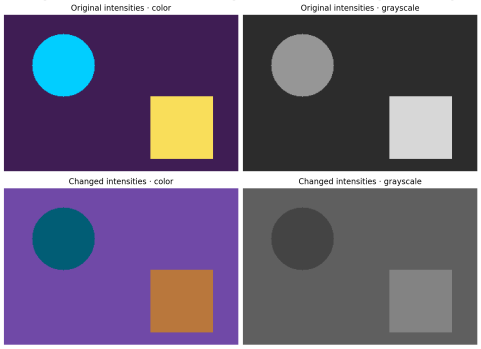
Hello Class,

As required by the discussion topic, I created a square and a circle on a colored background using OpenCV. The images are not much to look at, really. No shadows, textures, or other details that could interfere with the results. But there is an advantage to using an image I created myself. I know where the boundaries should be. For example, if Sobel produces a thick white outline around the square, I can recognize it, but that does not necessarily mean it identified the boundary correctly. I created the image in color at a resolution of 360 × 240 pixels, then converted it to grayscale. The square measures 96 × 96 pixels and the circle has a radius of 48. This time the circle is in the upper left, and the square in the lower right. I chose colors that produce grayscale values of 45 for the background, 215 for the square, and 150 for the circle. After that, I changed the values to 95, 132,

and 68, in the same order. I kept the objects exactly where they were; see Figure 1. I also added Gaussian noise (σ = 18) and repeated the noisy experiments ten times using fixed seeds.

Figure 1

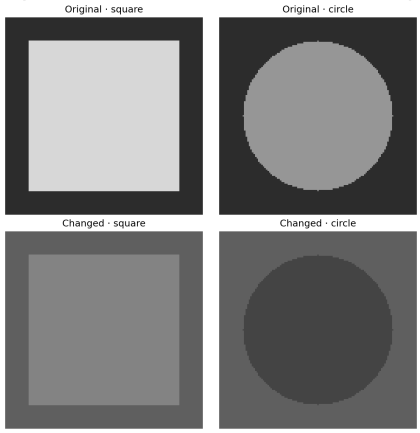
Original and Changed-Intensity Images: Color and Grayscale



Note. Both versions use the same shape positions. The color images are converted to grayscale before edge detection.

Figure 2

Square and Circle: Individual Grayscale Views



Note. The crops retain some background. All processing and scoring use the complete image.

I applied Gaussian smoothing with a 5 × 5 kernel (σ = 1.0) before running Canny, Sobel, and Laplacian. Smoothing helps reduce unwanted variations (OpenCV, n.d.-a).
For Sobel, I calculated gradients in the horizontal and vertical directions and combined them. Laplacian uses a second derivative to measure changes in that gradient. I used CV_64F so negative responses were not lost. For instance, going from dark to light can produce a different sign than going from light to dark. I took the magnitude afterward to keep both (OpenCV, n.d.-b).
Canny also calculates gradients, but uses non-maximum suppression to make the detected lines thinner, adding a few more steps. Then hysteresis, which is the process of following an edge through neighboring pixels rather than judging each one separately, uses two thresholds to keep strong edges and weaker ones that are connected to them (OpenCV, n.d.-a). I wanted to examine whether those extra steps would help when noise was added.

Initially, I set Canny to 80/160, Sobel to 110, and Laplacian to 50. I repeated the test with lower thresholds: 35/70, 60, and 25. My thinking behind applying these lower values was that this could

recover parts of the boundaries that had become difficult to detect when I reduced the contrast. But I realized there was a problem after running the tests with noise. The detectors seemed to pick up unwanted variations in brightness along with the actual edges, especially with the more sensitive settings. Canny uses two thresholds for hysteresis, while Sobel and Laplacian use different derivative responses, so their numbers are not directly comparable.

For scoring, I did not want to count every white pixel as a correct edge, as a thick white line could represent several detected pixels around one actual boundary. I used the known shape and location of the circle and square to define 760 pixels as the boundary reference. A detection could match one boundary pixel within 1.5 pixels, but neither pixel could be counted twice. Matches are true positives; extra detections are false positives; missed boundary points are false negatives. The precision metric measures how many of the detected pixels were correct, while recall measures how much of the actual boundary was recovered. I combined both as Boundary F1: 2 × precision × recall / (precision + recall).

To evaluate the edges in a different way, after researching online, I found Pratt's Figure of Merit, a method discussed by Abdou and Pratt (1979). It considers how far a detected edge is from its actual position, while also penalizing extra and missed detections. This is interesting because two detectors could produce similar outlines, but one might place its pixels a little farther from the real boundary. On the other hand, Martin et al. (2004) and Arbeláez et al. (2011) use precision-recall curves across several thresholds with boundaries that were labeled by people. Instead of judging one particular threshold, they examine how detection changes as the threshold is adjusted. In my case, I already knew where the square and circle were located, so I preferred to compare the detected edges directly against their boundaries, using Boundary F1 at the two threshold settings I tested.

Table 1

Boundary F1 Scores

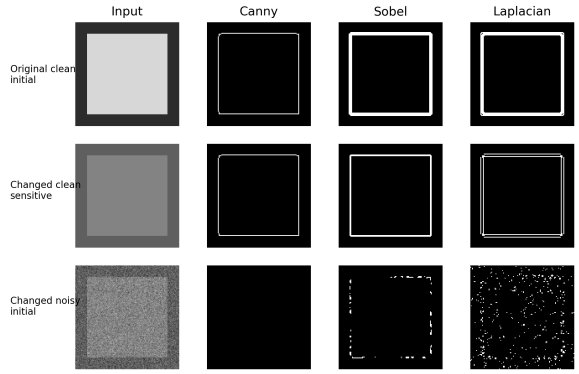
Image	Thresholds	Canny	Sobel	Laplacian
Original, clean	Initial	0.970	0.461	0.469
Original, clean	Sensitive	0.970	0.423	0.335
Changed intensity, clean	Initial	0.000	0.000	0.010
Changed intensity, clean	Sensitive	0.965	0.738	0.569
Original intensity + noise	Initial	0.980	0.462	0.294
Original intensity + noise	Sensitive	0.317	0.296	0.063
Changed intensity + noise	Initial	0.094	0.414	0.157
Changed intensity + noise	Sensitive	0.292	0.390	0.064

Note. Higher scores mean closer agreement with the known boundaries using the 1.5-pixel allowance. Noisy scores are averages of ten runs.

The table data shows that Canny performed better on the original clean image, scoring 0.970, although Sobel and Laplacian matched all 760 reference pixels; their scores were only 0.461 and 0.469. So, why is that? Well, Sobel produced 2,536 white pixels and Laplacian 2,480. Most of the extras were around the boundaries. Canny produced only 716 pixels, all matched, although it missed 44 reference points. Figure 3 makes this easier to see. The square is recognizable in all three results, but Sobel's edges are thicker, while Laplacian adds responses near the corners.

Figure 3

Square: Canny, Sobel, and Laplacian



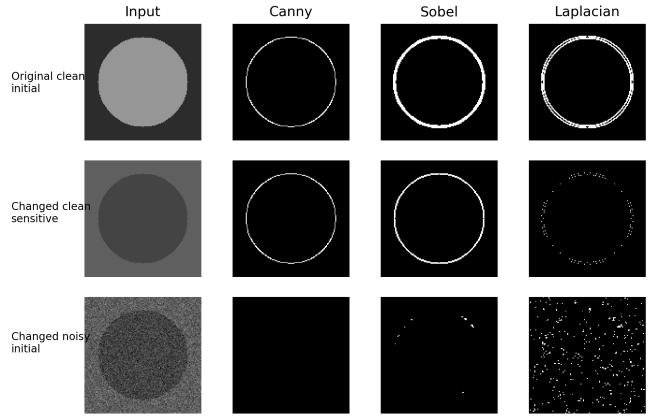
Note: Rows show original clean/initial, changed clean/sensitive, and changed noisy/initial. The noisy row uses seed 51501. Detection was performed before cropping.

The circle was more difficult after reducing the contrast. With noise and the initial thresholds, Sobel recovered about 49% of the square's boundary but only 5% of the circle's, averaged across ten trials. Their contrasts were 37 and 27 grayscale levels. Both received the same processing. The program does not know that one is a square and the other a circle; it examines their pixel intensities.

Figure 4 shows Laplacian's nearly double outline around the original circle and scattered responses after changing the intensities. Canny's low-contrast noisy result is black, although the circle is still visible in the input.

Figure 4

Circle: Canny, Sobel, and Laplacian



Note: The rows match Figure 3. They show one noisy trial, not the ten-trial average.

I would choose Canny for the clean images, provided the thresholds suit their contrast. It scored 0.970 on the original and 0.965 after changing the intensities with lower thresholds. It also did best with noise at the original intensities (0.980). When I reduced contrast and added noise, Sobel scored higher at both settings, 0.414 and 0.390. None did well on the clean low-contrast image with the initial thresholds. Laplacian's 0.010 was technically highest, but it matched only four boundary pixels. That was not a useful detection.

-Alex

References

Abdou, I. E., & Pratt, W. K. (1979). Quantitative design and evaluation of enhancement/thresholding edge detectors. *Proceedings of the IEEE*, *67*(5), 753–763. https://doi.org/10.1109/PROC.1979.11325

Arbeláez, P., Maire, M., Fowlkes, C., & Malik, J. (2011). Contour detection and hierarchical image segmentation. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, *33*(5), 898–916. https://doi.org/10.1109/TPAMI.2010.161

Martin, D. R., Fowlkes, C. C., & Malik, J. (2004). Learning to detect natural image boundaries using local brightness, color, and texture cues. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, *26*(5), 530–549. https://doi.org/10.1109/TPAMI.2004.1273918

OpenCV. (n.d.-a). *Canny edge detection*. OpenCV documentation. https://docs.opencv.org/4.13.0/da/d22/tutorial_py_canny.html

OpenCV. (n.d.-b). *Image gradients*. OpenCV documentation. https://docs.opencv.org/4.13.0/d5/d0f/tutorial_py_gradients.html